PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

Email: umhackathon@um.edu.my

Table of Contents

Email: umhackathon@um.edu.my

Project Name: AgriMind - AI-powered decision intelligence for urban vertical farming

Version: 1.0

1. Project Overview

- **Problem Statement:** In response to food security challenges, land scarcity, and climate change, urban vertical farming is rapidly expanding in Malaysia. However, most small and medium-sized urban farm operators lack access to smart decision-support systems. Farmers are forced to rely on fixed irrigation and lighting schedules, manual observation to detect pests and diseases, and nutrient management based on intuition.

  This results in significant resource waste, with fertilizer wastage exceeding 30% and electricity wastage reaching 20%, while also leading to delayed responses to deteriorating crop health and water quality threats (particularly during Malaysia's rainy season), as well as inefficient energy utilization during TNB's peak electricity pricing periods. Existing agricultural software tools are either too expensive, too generic, or designed specifically for outdoor field cultivation, leaving the urban vertical farming sector underserved.

- **Target Domain:** Urban Precision Agriculture. specifically referring to Malaysia's indoor hydroponic vertical farming projects, targeting small to medium-sized farm operators in the Kuala Lumpur, Johor, and Selangor regions.

- **Proposed Solution Summary:** AgriMind — AI-Powered Decision Intelligence for Urban Vertical Farming

  AgriMind is an AI-enhanced farm monitoring dashboard that uses Google Gemini API as its core inference engine. It continuously monitors farm sensor data, interprets environmental conditions, and generates explainable, context-aware cultivation recommendations. Unlike traditional automation systems that operate on fixed rules, AgriMind reasons through trade-offs by balancing crop health, energy costs, and risk mitigation, then communicates each decision to farm operators in plain language. This prototype focuses on real-time data visualization, scenario simulation, and AI-generated recommendations, with automatic hardware control planned for future releases.

2. Background & Business Objective

- **Background of the Problem:** Driven by government initiatives such as the National Agri-Food Policy 2021–2030 and concerns over urban food security, Malaysia's urban agriculture sector has seen significant growth over the past decade. However, a substantial technological gap remains between large-scale commercial farms and small-scale urban growers.

  Through research and stakeholder analysis, the following key pain points were identified:
  - **Resource Waste:** Fixed irrigation and lighting schedules, regardless of actual plant needs or electricity prices, lead to resource waste.
  - **Detection Delays:** Manual visual inspections are too slow to promptly identify early-stage fungal infections, nutrient deficiencies, or water contamination issues.
  - **Pollution risks:** During Malaysia's rainy season, upstream river pollution and flash floods can destroy entire pond or tank systems before operators have a chance to react.

- **Importance of Solving This Issue:** Addressing these issues offers direct and quantifiable economic benefits for urban farm operators in Malaysia:
    - Reducing chemical fertilizer waste by 30% directly cuts one of the largest operational costs.
    - Reducing electricity waste by 20% through peak-hour scheduling results in significant monthly savings.
    - Avoiding even a single instance of crop yield loss (each loss valued at 5,000 to 20,000 ringgit) through early risk mitigation is sufficient to justify the cost of the entire system.
    - With AI-guided support, even non-professional operators can manage farms, making precision agriculture accessible to Malaysia's B40 population and small business communities.

- **Strategic Fit / Impact:** AgriMind aligns with Malaysia's National Food Security Policy and the United Nations Sustainable Development Goal 2 (Zero Hunger). By integrating Google Gemini API as its core inference engine, the system demonstrates a modern AI technology stack, making it accessible to a broader community of Malaysian farmers. The subscription-based SaaS model also provides the system with a scalable commercial path that extends beyond hackathons.

3. Product Purpose

3.1. Main Goal of the System: To build an AI-powered decision intelligence layer on top of farm sensor data, enabling urban vertical farm operators to make smarter, faster, and more cost-effective decisions without requiring specialised agricultural expertise. For the prototype, the focus is on real-time monitoring, scenario simulation, and AI-generated recommendations, with automated hardware actuation planned for future versions.

3.2. Intended Users (Target Audience):

User Type	Description	Primary Need
Urban Farm Operator	Owners of small-medium scale hydroponic farms in Malaysia	Automated daily decisions and alerts
Farm Technician	On-site staff responsible for managing daily farm operations	Real-time sensor monitoring and guidance
Farm Investor / Owner	Business owner tracking ROI and operational costs	Monthly performance reports and savings

4. System Functionalities

4.1. Description: AgriMind operates as an AI-enhanced farm monitoring dashboard. It collects structured sensor data from Arduino-based IoT hardware (water level sensor + DHT11) and simulated sensor sources (pH, EC, NPK, light intensity) , combines this data with optional weather context, and passes context-relevant prompts to Google Gemini API for inference. The system then outputs interpretable decision results through the dashboard. For the prototype, all recommendations are displayed for operator action, automatic hardware control is planned for future releases.

4.2. Key Functionalities:

- **AI Proactive Recommendation:** AgriMind's Gemini-powered AI system analyzes current sensor data, simulated crop conditions, and weather context to provide proactive recommendations on demand. When the operator selects a scenario or connects to live Arduino data, the system generates actionable advice to help optimize yield, reduce costs, and prevent crop stress. In this prototype, recommendations are generated manually via the "Generate Gemini AI Insight" button, not continuously.

- **Predictive Risk Protection:** AgriMind monitors sensor data for abnormal conditions. When simulated or real sensor readings exceed safety thresholds (e.g., temperature > 38°C, EC > 3.5 mS/cm, soil moisture < 20%), the system displays an alert with a clear AI-generated recommendation for operator action. For the prototype, the system does not automatically control water valves or pumps; the operator must act on recommendations.

- **Explainable AI Decision Logs:** Every Gemini-generated decision is displayed on the dashboard with a clear, plain-language explanation and a list of key influencing factors. For example, the system explains why "Activate Emergency Cooling" is recommended when temperature exceeds 38°C, listing factors such as "Heat Stress," "Stomatal Closure," and "Wilting Risk." All decisions are logged in the System Logs view with timestamps for auditability.

- **Quantifiable Business Impact Tracking:** AgriMind's business intelligence module compares AI-optimized operations with traditional fixed-schedule farming and generates monthly economic impact reports. It tracks fertilizer savings, electricity cost reduction, lower crop losses, and increased revenue from better yield and quality, providing clear evidence of return on investment.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: AgriMind uses Google Gemini API as its core large language model (LLM) due to its powerful reasoning capabilities, ability to generate structured JSON output, and cost-effective pricing for prototyping. The model enables multi-step trade-off analysis (e.g., balancing energy consumption with crop growth) and generates structured recommendations that the dashboard can render directly.

4.3.2. Prompting Strategy: AgriMind employs a multi-step agent prompting strategy:

- **Step 1 – Context Injection:** The system injects the current scenario name, crop type (Lettuce Zone A), and sensor thresholds.
- **Step 2 – Sensor Data Injection:** Real-time sensor readings (temperature, humidity, soil moisture, light, pH, EC, NPK) are formatted into JSON and injected.
- **Step 3 – Weather Context:** Optional weather data from OpenWeather API is appended for environmental awareness.
- **Step 4 – Structured Output:** The response is constrained to a JSON schema: { recommendation, explanation, impact }.

This strategy was chosen because farm decisions are inherently multivariate. For example, a single metric such as elevated electrical conductivity (EC) can have different implications depending on temperature, crop growth stage, and recent weather conditions. Chain-of-Thought Reasoning enables the Gemini to comprehensively consider all variables before taking action.

4.3.3. Context & Input Handling:

Input Type	**Max Size**	**Handling Strategy**
Sensor data JSON	~500 tokens	Direct injection; refreshed when user triggers Gemini insight
Scenario context	200 characters	Injected as part of prompt (e.g., "Heat Stress", "Over-Irrigation")
Weather context	~300 tokens	Fetched from OpenWeather API and appended when available

4.3.4. Fallback & Failure Behavior:

- **Schema validation failed:** The backend logs the error and returns a fallback recommendation.
- **API unavailable:** The system returns a cached or rule-based recommendation and notifies the operator.
- **Logging:** All Gemini responses are logged with timestamps for auditing.

5. User Stories & Use Cases

- **User Stories:** As a farm operator, I want to receive an instant AI recommendation when sensor readings exceed normal ranges, so I can take corrective action before my crops are damaged.

- **Use Cases (Main Interactions):**

    1. The operator selects a scenario (e.g., "Heat Stress") or connects to live Arduino data.
    2. The system displays current sensor readings and an "AI Directive" card.
    3. Operator clicks "Generate Gemini AI Insight".
    4. Backend calls Google Gemini API with sensor data and weather context.
    5. Dashboard displays AI recommendation, explanation, key influencing factors, and predicted business impact.
    6. Operators can act on the recommendation manually (automatic actuation is future scope).

6. Features Included (Scope Definition)

Feature	Description
Real-Time Sensor Dashboard	Displays live readings from Arduino (temperature, humidity, soil moisture) plus simulated values for unavailable sensors
Gemini AI Decision Engine	Core inference loop: sensor data → Gemini analysis → structured output → dashboard display
Explainable Decision Log	All AI decisions include plain-language rationale, timestamps, and key influencing factors
Scenario Simulation	Five predefined scenarios to demonstrate AI behaviour under different stress conditions
Business Impact Projections	Estimated savings in water, fertiliser, and yield improvement vs. non-AI baseline
System Logs Viewer	Filterable event logs from backend operations
Arduino Integration (Live Mode)	Reads real sensor data from Arduino via HTTP endpoint (water level + DHT11)

7. Features Not Included (Scope Control)

- Automatic water valve switching
- Full commercial IoT deployment across multiple farm locations
- Multi-user role-based access control with enterprise authentication
- Direct integration with Malaysian agricultural trading platforms
- Real-time video streaming from farm cameras

8. Assumptions & Constraints

- **LLM Cost Constraint:**

Item	Estimate / Decision
Average tokens per sensor analysis call	~800 tokens (input) + ~400 tokens (output) = ~1,200 tokens
Sensor read frequency	Every 2 seconds for live mode (prototype); every 60 seconds in production
Daily token usage estimate	~1.7M tokens/day per farm
Cost optimisation — caching	Identical sensor patterns within ±5% are cached for 10 minutes to avoid redundant GLM calls
Cost optimisation — batching	Non-critical decisions (daily summary, weekly reports) are batched overnight during off-peak compute time

- **Technical Constraints:**
  - **Arduino integration:** Live mode requires the backend (localhost:3000) to be running and Arduino to send data via Serial-to-HTTP bridge.
  - **Dashboard:** Optimised for desktop browsers; mobile compatibility in progress.
  - **External APIs:** Requires active internet connection for Gemini API and OpenWeather API.

- **Performance Constraints:**
  - **AI decision latency:** 5–10 seconds for Gemini API round trip.
  - **Sensor polling:** Frontend polls backend every 2 seconds for live mode.
  - **Simulated sensors:** Light, pH, EC, NPK values are simulated when not available from hardware.

- **User Input:** The AI recommendation engine requires the operator to manually trigger analysis by clicking the "Generate Gemini AI Insight" button on the Scenario Lab page. The system will generate a decision based on:
    - Current sensor readings (live Arduino data if in "Arduino Live" mode, or simulated scenario data)
    - Optional weather context (fetched from OpenWeather API when available)

  If no sensor data is available (e.g., backend not running, Arduino not connected, or no scenario selected), the dashboard will:
    - Display default or fallback sensor values
    - Show a "Backend unavailable" warning in browser console
    - The Gemini button will still attempt to call the API, but may return an error message

9. Risks & Questions Throughout Development

- **Design Similarity:** Currently, only Zone A (Lettuce) is implemented. The prompt includes crop type and scenario name (e.g., Heat Stress, Over-Irrigation) to ensure accurate, context-specific responses. Future multi-zone support will use separate prompts for each crop type.

- **Frame Stability:** The prototype uses manual AI triggers instead of continuous automatic analysis, preventing log overload. System Logs refresh every 5 seconds and automatically remove older entries, keeping the dashboard stable and organized.

- **Drawing Interpretation:** Sensor values are clamped to valid ranges before being sent to the AI (e.g., temperature 0–50°C, humidity 0–100%). Invalid Arduino readings are logged as warnings, while the dashboard shows status alerts like “DANGER” or “WARN.” Future improvements will include anomaly detection for hardware failures.